

Rheinisch-Westfälische Technische Hochschule Aachen

Bachelor Thesis

Outermost Almost-Sure Termination of Probabilistic Term Rewrite Systems

by

Lucas Ribeiro da Silva

Advisor: Jan-Christoph Kassing

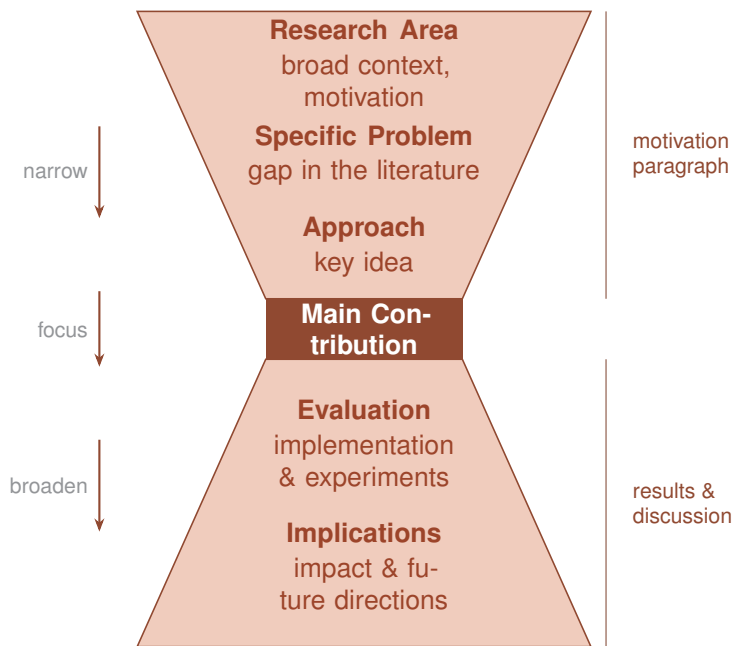
Lehr- und Forschungsgebiet Informatik 2
Programming Languages and Verification

First Examiner:
Prof. Dr. Examiner 1

Second Examiner:
Prof. Dr. Examiner 2

August 11, 2026

Abstract



Hourglass structure of a good abstract. Start broad (research area and problem), funnel down to the specific contribution, then widen again to state results and implications.

Write a self-contained summary of roughly 150–250 words following the hourglass structure above. Cover the following five points in order:

1. **Research area.** Name the field (e.g., automated termination analysis, probabilistic verification, complexity analysis of rewrite systems) and state in one sentence what the field studies and why it matters.
2. **Specific problem.** State precisely which problem this thesis addresses and why it is challenging or not yet fully solved by existing approaches.
3. **Approach.** Describe the key idea of your contribution in one or two sentences. Avoid mathematical notation and abbreviations, and use informal language.
4. **Theoretical results.** Name the main property proved (soundness, completeness, an equivalence result, or a complexity bound) and state it informally.
5. **Implementation and evaluation (if applicable).** Name the tool and the benchmark set (e.g., the Termination Problems Data Base) and give the headline result: how many more examples are solved compared to the baseline and what is the runtime overhead.

-
6. **Implications.** Explain how your results solved the issue or filled the gap you pointed out beforehand.

Acknowledgements

I want to express my most profound appreciation to my supervisor!
Maybe I want to thank someone else as well...

Contents

Abstract	ii
Acknowledgements	iv
Contents	v
1 Introduction	1
1.1 Motivation and Problem Statement	1
1.2 Related Work	2
1.3 Contribution	3
1.4 Structure of the Thesis	3
2 Preliminaries	4
2.1 Relations and Terms	4
2.2 Term Rewrite Systems and Strategies	6
2.3 Probabilistic Term Rewrite Systems	7
2.4 Probabilistic Derivations	8
2.5 Probabilistic Termination	9
2.6 Transformation-Based Strategy Transfer	11
3 [Title of Your Main Contribution]	12
3.1 [Central Definitions / The New Framework]	12
3.2 [Main Results]	12
4 Evaluation	14
4.1 Implementation	14
4.2 Experimental Setup	14
4.3 Results	15
4.4 Discussion	15
5 Conclusion	16
Bibliography	17

Introduction

Term rewriting provides a small but expressive model of computation. A term rewrite system describes a computation by rules that replace matching subterms, and a computation is obtained by applying such rules repeatedly. Termination asks whether this process can continue forever. Besides being a fundamental property in its own right, termination is a prerequisite for using rewrite systems as executable specifications and for reasoning about the result of a computation. It has therefore been studied extensively, both as a mathematical problem and as a target of automated verification; see, for example, [2].

Probabilistic term rewrite systems (PTRSs) extend this model by allowing a rule to choose among several right-hand sides according to a probability distribution. They can represent randomised algorithms and probabilistic program fragments while retaining the syntax and compositional structure of term rewriting. For such systems, classical termination is often unnecessarily strong. A probabilistic computation may admit an infinite run and nevertheless terminate with probability one. This motivates *almost-sure termination* (AST), which requires the probability of eventually reaching a normal form to be one [1]. Importantly, AST is a statement about probability mass rather than about every individual path.

1.1 Motivation and Problem Statement

The order in which redexes are contracted can change termination behaviour. A redex is an occurrence of a subterm that matches the left-hand side of a rule. Under *outermost* rewriting, a redex may be contracted only if it is not strictly contained in another redex. Under *innermost* rewriting, the chosen redex may not contain another redex. These strategies correspond to two opposite evaluation disciplines: outermost rewriting gives priority to the surrounding computation, whereas innermost rewriting evaluates arguments first.

The difference is illustrated by the PTRS in Figure 1.1. Its producer constructs a pair whose first component is chosen probabilistically and whose second component invokes the producer again. Starting with a projection of the producer, outermost evaluation discards the recursive component and returns the first component. Innermost evaluation must instead keep expanding the recursive component before it may apply the projection. The same start term therefore terminates under outermost rewriting but not under innermost rewriting.

A formal account using lifted probabilistic derivations is given in Example 2.3.4.

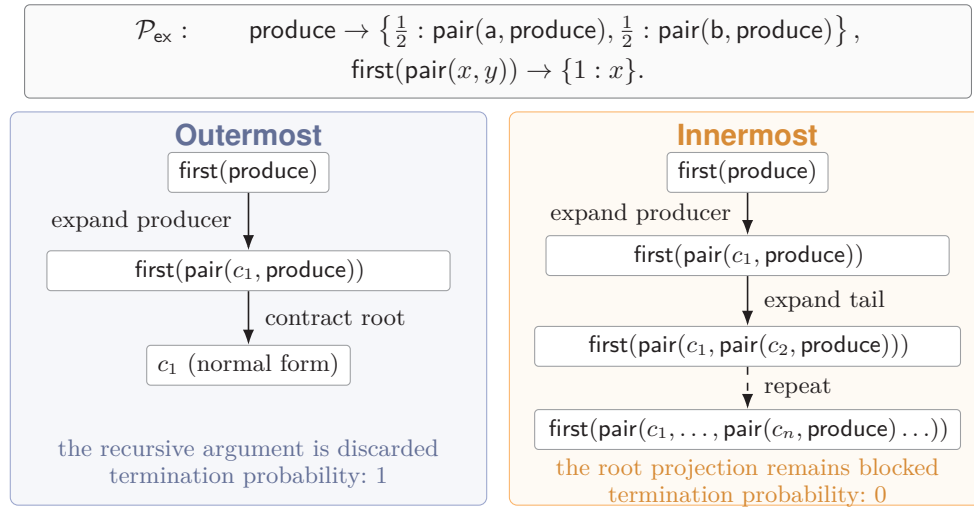


Figure 1.1: Strategy-dependent behaviour from $\text{first}(\text{produce})$. Each c_i is independently chosen as $\mathbf{a}$ or $\mathbf{b}$, each with probability $\frac{1}{2}$.

Automated proof techniques are especially well developed for innermost rewriting. This suggests reducing an outermost termination problem to an innermost one. The central question of this thesis is therefore:

Can outermost almost-sure termination of a PTRS be established by transforming it into a PTRS whose innermost almost-sure termination can be analysed instead?

The intended transformation must do more than reproduce the set of reachable terms. Soundness requires it to preserve the probabilities of the original alternatives and to represent nondeterministic choices between redexes and rules. Completeness additionally requires control over computations beginning with arbitrary terms over the extended signature, including administrative configurations. Such computations must not introduce positive non-termination probability that is absent from the source system. The essential new issue in the probabilistic setting is therefore the preservation of probability mass, rather than merely the existence of infinite reductions.

1.2 Related Work

Transformation-based proofs of strategy-sensitive termination have a history in ordinary term rewriting. Raffelsieper and Zantema transform an outermost termination problem into an unrestricted termination problem. Their principal transformation is sound for outermost ground termination and is effectively constructible as a finite TRS for finite quasi-left-linear systems [4]. Thiemann later introduced a different marker transformation for finite TRSs and proved the exact correspondence

$$\mathcal{R} \text{ is outermost terminating} \iff \mathcal{R}' \text{ is innermost terminating.}$$

This construction is the deterministic starting point for the transformation studied in this thesis [5].

Probabilistic rewriting requires a separate semantic treatment. Avanzini, Dal Lago, and Yamada give a systematic account of PTRSs and probabilistic termination notions [1].

Kassing and Giesl study when innermost results for a PTRS can be transferred to unrestricted probabilistic rewriting, and show that additional syntactic restrictions are needed because duplication can affect probability mass [3]. Their work concerns innermost versus full rewriting; it does not provide the outermost-to-innermost transformation considered here.

The deterministic transformation suggests the construction, but its correctness proof does not transfer automatically to PTRSs. In a deterministic system, one infinite reduction is enough to refute termination. In a probabilistic system, the existence of infinite paths is compatible with AST when the total probability of infinite computations is zero. Consequently, correctness has to be proved for infinite lifted derivations, or equivalently for fully evaluated rewrite sequence trees, while retaining their branch probabilities.

1.3 Contribution

This thesis adapts the marker-based outermost-to-innermost transformation of [5] to probabilistic term rewrite systems. Given a PTRS $\mathcal{P}$, the construction produces a PTRS $\mathcal{P}'$ over an extended signature. Fresh administrative symbols guide an innermost computation to positions that represent permitted outermost steps of $\mathcal{P}$. The rules that move these symbols are deterministic; probabilistic branching occurs only when an original rule is simulated.

The main contributions are:

1. a probabilistic outermost-to-innermost transformation, including the extended signature and its administrative rewrite rules;
2. a simulation result showing that every outermost probabilistic step is represented by a finite, non-empty sequence of innermost steps while preserving the probabilities of all outcomes; and
3. a lifting of the simulation and projection arguments to infinite probabilistic derivations, yielding the sound-and-complete equivalence

$$\text{AST}(\xrightarrow{\circ}_{\mathcal{P}}) \iff \text{AST}(\xrightarrow{\circ}_{\mathcal{P}'}).$$

Both sides quantify over all terms, including non-ground terms. In particular, the target property concerns every term over the extended signature, not only terms produced by the designated source encoding. The result established here concerns AST; no corresponding claim for PAST or SAST is made.

1.4 Structure of the Thesis

The remainder of this thesis is organised as follows. Chapter 2 introduces terms, rewrite strategies, finite multi-distributions, probabilistic rewriting, rewrite sequence trees, and the termination notions used throughout the thesis. It also recalls the deterministic transformation result on which the new construction is based. Chapter 3 presents the probabilistic transformation and proves its simulation and correctness properties. Chapter 4 illustrates the construction on representative examples and discusses its applicability and limitations. Finally, Chapter 5 summarises the results and outlines possible extensions.

CHAPTER 2

Preliminaries

This chapter fixes the notation and semantic conventions used in the remainder of the thesis. Section 2.1 introduces relations, terms, positions, substitutions, and contexts. Section 2.2 defines term rewrite systems and distinguishes full, outermost, and innermost rewriting. Section 2.3 extends these notions to probabilistic term rewrite systems. The lifted semantics and rewrite sequence trees are presented in Section 2.4, followed by the definitions of AST, PAST, and SAST in Section 2.5. Finally, Section 2.6 recalls the deterministic transformation that forms the starting point of this thesis. Standard background on term rewriting can be found in [2]; the probabilistic conventions follow [1, 3].

2.1 Relations and Terms

We begin with notation for relations. It will be used both for ordinary rewrite relations and for relations between probability distributions. Throughout the thesis, $\mathbb{N} = \{0, 1, 2, \dots\}$.

Definition 2.1.1: Relations, closures, and normal forms

Let A be a set and let $\rightarrow \subseteq A \times A$ be a binary relation. For $n \in \mathbb{N}$, its n -fold composition $\rightarrow^n$ is defined by

$$a \rightarrow^0 b \iff a = b$$

and

$$a \rightarrow^{n+1} b \iff \text{there is a } c \in A \text{ with } a \rightarrow^n c \text{ and } c \rightarrow b.$$

The relations

$$\rightarrow^* = \bigcup_{n \in \mathbb{N}} \rightarrow^n \quad \text{and} \quad \rightarrow^+ = \bigcup_{n \geq 1} \rightarrow^n$$

are the reflexive-transitive and transitive closures of $\rightarrow$, respectively. An element $a \in A$ is a *normal form* if no $b \in A$ satisfies $a \rightarrow b$. The set of all normal forms is denoted by

$$\text{NF}_{\rightarrow} = \{a \in A \mid \nexists b \in A. a \rightarrow b\}.$$

The relation $\rightarrow$ is *terminating*, or *well-founded in the forward direction*, if there is no infinite sequence $a_0 \rightarrow a_1 \rightarrow a_2 \rightarrow \dots$.

A term is a finite tree whose internal nodes are function symbols and whose leaves may additionally be variables.

Definition 2.1.2: Signatures and terms

A *signature* $\Sigma = \uplus_{n \in \mathbb{N}} \Sigma_n$ is a finite set of function symbols with fixed arities; a symbol $f \in \Sigma_n$ has arity n . Let $\mathcal{V}$ be a countably infinite set of variables disjoint from Σ . The set $\mathcal{T}(\Sigma, \mathcal{V})$ of terms over Σ and $\mathcal{V}$ is the least set satisfying:

- every $x \in \mathcal{V}$ is in $\mathcal{T}(\Sigma, \mathcal{V})$; and
- if $f \in \Sigma_n$ and $t_1, \dots, t_n \in \mathcal{T}(\Sigma, \mathcal{V})$, then $f(t_1, \dots, t_n) \in \mathcal{T}(\Sigma, \mathcal{V})$.

Constants are the symbols of arity zero. A term without variables is *ground*; the set of ground terms is denoted by $\mathcal{T}(\Sigma, \emptyset)$. For a term t , $\mathcal{V}(t)$ is the set of variables occurring in t . If $t = f(t_1, \dots, t_n)$, then $\text{root}(t) = f$.

For example, with binary **pair**, unary **first**, and constants **a** and **produce**, the expression **first(pair(a, produce))** is a ground term.

Positions identify nodes in the tree representation of a term. We use the empty sequence ε for the root and concatenate position components with a dot.

Definition 2.1.3: Positions, subterms, and replacement

The set of positions of a term is defined recursively by

$$\text{Pos}(x) = \{\varepsilon\}$$

for variables x , and

$$\text{Pos}(f(t_1, \dots, t_n)) = \{\varepsilon\} \cup \{i.\pi \mid 1 \leq i \leq n, \pi \in \text{Pos}(t_i)\}.$$

For $\pi \in \text{Pos}(t)$, the subterm of t at π is denoted by $t|_\pi$. Replacing this subterm by s yields the term $t[s]_\pi$.

For positions π and τ , we write $\pi \leq \tau$ if π is a prefix of τ , and $\pi < \tau$ if the prefix is proper. In this case, π is *above* τ and τ is *below* π . Positions are *parallel*, written $\pi \parallel \tau$, if neither is a prefix of the other.

For $t = \text{first}(\text{pair}(\mathbf{a}, \text{produce}))$, we have $t|_{1.2} = \text{produce}$ and

$$t[\mathbf{b}]_{1.2} = \text{first}(\text{pair}(\mathbf{a}, \mathbf{b})).$$

Moreover, $\varepsilon < 1 < 1.2$.

Definition 2.1.4: Substitutions and contexts

A *substitution* is a mapping $\sigma : \mathcal{V} \rightarrow \mathcal{T}(\Sigma, \mathcal{V})$ that differs from the identity on only finitely many variables. It extends homomorphically to terms:

$$f(t_1, \dots, t_n)\sigma = f(t_1\sigma, \dots, t_n\sigma).$$

We call $t\sigma$ an *instance* of t .

A *context* C is a term containing exactly one occurrence of a fresh hole symbol $\square$. The term obtained by replacing the hole with t is denoted by $C[t]$.

Thus, a term s occurs as a subterm of t exactly when $t = C[s]$ for a suitable context C .

2.2 Term Rewrite Systems and Strategies

Definition 2.2.1: Term rewrite systems

A *rewrite rule* is a pair $\ell \rightarrow r$ of terms such that $\ell \notin \mathcal{V}$ and $\mathcal{V}(r) \subseteq \mathcal{V}(\ell)$. A *term rewrite system* (TRS) $\mathcal{R}$ is a finite set of rewrite rules.

The root symbols of left-hand sides are the *defined symbols* of $\mathcal{R}$. The remaining symbols of the signature are called *constructors*.

The variable condition prevents a rewrite step from introducing arbitrary new terms through previously unbound variables.

Definition 2.2.2: Rewrite steps and redexes

Let $s, t \in \mathcal{T}(\Sigma, \mathcal{V})$. We write

$$s \xrightarrow{\mathcal{R}, \pi} t$$

if there are a rule $\ell \rightarrow r \in \mathcal{R}$ and a substitution σ such that

$$s|_{\pi} = \ell\sigma \quad \text{and} \quad t = s[r\sigma]_{\pi}.$$

The occurrence $s|_{\pi} = \ell\sigma$ is a *redex*, and π is a *redex position*. We omit π when its value is irrelevant and write $s \xrightarrow{\mathcal{R}} t$.

The next definition restricts which redex positions may be selected. Stating the conditions in terms of redex positions, rather than the eventual target term, is important: a position is disallowed whenever any rule is applicable strictly above or below it.

Definition 2.2.3: Outermost and innermost rewriting

Let $\text{Red}_{\mathcal{R}}(s)$ be the set of redex positions in s . A step $s \xrightarrow{\mathcal{R}, \pi} t$ is

- *outermost*, written $s \xrightarrow{\circ, \pi} t$, if there is no $\tau \in \text{Red}_{\mathcal{R}}(s)$ with $\tau < \pi$; and
- *innermost*, written $s \xrightarrow{\cdot, \pi} t$, if there is no $\tau \in \text{Red}_{\mathcal{R}}(s)$ with $\pi < \tau$.

We write $\xrightarrow{\mathcal{R}}$ for unrestricted, or *full*, rewriting and omit positions from $\xrightarrow{\circ, \mathcal{R}}$ and $\xrightarrow{\cdot, \mathcal{R}}$ when they are not needed.

Equivalently, an innermost step contracts a redex whose proper subterms are all normal forms. Every reducible finite term has both an outermost and an innermost redex. Consequently, full, outermost, and innermost rewriting have the same normal forms, although their infinite derivations can differ.

Definition 2.2.4: Strategy-specific termination

A TRS $\mathcal{R}$ is *outermost terminating* if $\xrightarrow{\circ, \mathcal{R}}$ is terminating, and it is *innermost terminating* if $\xrightarrow{\cdot, \mathcal{R}}$ is terminating. Full termination is termination of $\xrightarrow{\mathcal{R}}$.

A term is *linear* if every variable occurs at most once. A TRS is left-linear (right-linear) if every left-hand side (right-hand side) is linear, and it is linear if both conditions hold. These properties are not required by the deterministic transformation of [5], but they occur in related strategy-transfer criteria and will be mentioned when those criteria are compared.

2.3 Probabilistic Term Rewrite Systems

Probabilistic rules return finite multi-distributions as in [1, 3]. The prefix “multi” records that two alternatives may lead to syntactically equal terms while still being retained as separate occurrences. This is convenient when individual branches are mapped by a simulation proof.

Definition 2.3.1: Finite multi-distributions

Let A be a non-empty set. A *finite multi-distribution* over A is a finite multiset

$$\mu = \{p_1 : a_1, \dots, p_k : a_k\}$$

with $a_i \in A$, $0 < p_i \leq 1$, and $\sum_{i=1}^k p_i = 1$. The set of all finite multi-distributions over A is denoted by $\text{FDist}(A)$, and $\text{Supp}(\mu)$ denotes the multiset of elements carrying positive weight.

For $0 < q \leq 1$, scaling is defined by

$$q \cdot \mu = \{qp_1 : a_1, \dots, qp_k : a_k\}.$$

If $F : A \rightarrow B$ is a function, its pointwise lifting to multi-distributions is

$$F_{\#}(\mu) = \{p_1 : F(a_1), \dots, p_k : F(a_k)\}.$$

The scaled object $q \cdot \mu$ is a weighted multiset of total weight q , not in general an element of $\text{FDist}(A)$. Such objects are combined with multiset union so that the resulting weights again sum to one.

Definition 2.3.2: Probabilistic term rewrite systems

A *probabilistic rewrite rule* has the form

$$\ell \rightarrow \mu, \quad \mu \in \text{FDist}(\mathcal{T}(\Sigma, \mathcal{V})),$$

where $\ell \notin \mathcal{V}$ and $\mathcal{V}(r) \subseteq \mathcal{V}(\ell)$ for every $r \in \text{Supp}(\mu)$. A *probabilistic term rewrite system* (PTRS) $\mathcal{P}$ is a finite set of probabilistic rewrite rules. We write a rule explicitly as

$$\ell \rightarrow \{p_1 : r_1, \dots, p_k : r_k\}.$$

An ordinary rewrite rule $\ell \rightarrow r$ is identified with the deterministic probabilistic rule $\ell \rightarrow \{1 : r\}$.

PTRSs may be defined more generally as countable sets of rules. This thesis restricts attention to finite PTRSs over finite signatures, as required for the transformation to produce a finite system.

One step still makes a nondeterministic choice of rule and redex position. Once that choice has been made, the right-hand side determines the probabilistic outcomes.

Definition 2.3.3: Probabilistic rewrite steps

Let $\ell \rightarrow \{p_1 : r_1, \dots, p_k : r_k\} \in \mathcal{P}$. If $s|_{\pi} = \ell\sigma$, then s rewrites at π to the multi-distribution

$$\mu = \{p_1 : s[r_1\sigma]_{\pi}, \dots, p_k : s[r_k\sigma]_{\pi}\}.$$

This unrestricted step is written $s \xrightarrow{\mathcal{P}, \pi} \mu$. It is written $s \xrightarrow{\circ}_{\mathcal{P}, \pi} \mu$ if π is an outermost redex position and $s \xrightarrow{\iota}_{\mathcal{P}, \pi} \mu$ if π is an innermost redex position, using the strategy conditions from Section 2.2.

Normal forms and redex positions depend only on the left-hand sides of $\mathcal{P}$; the probabilities influence the successors, but not where a rule is applicable. We therefore write $\text{Red}_{\mathcal{P}}(s)$ for the positions at which a left-hand side of $\mathcal{P}$ matches a subterm of s .

Example 2.3.4 (Running example). Let $\mathcal{P}_{\text{ex}}$ contain the rules

$$\begin{aligned} \text{produce} &\rightarrow \left\{ \frac{1}{2} : \text{pair}(\text{a}, \text{produce}), \frac{1}{2} : \text{pair}(\text{b}, \text{produce}) \right\}, \\ \text{first}(\text{pair}(x, y)) &\rightarrow \{1 : x\}. \end{aligned}$$

The first probabilistic step from $\text{first}(\text{produce})$ is both outermost and innermost, since the occurrence of produce is the only redex. It yields

$$\left\{ \frac{1}{2} : \text{first}(\text{pair}(\text{a}, \text{produce})), \frac{1}{2} : \text{first}(\text{pair}(\text{b}, \text{produce})) \right\}.$$

In each outcome, the root redex is outermost, while the nested produce redex is innermost. Thus the strategy determines which redex must be selected next.

2.4 Probabilistic Derivations

A term-to-distribution relation describes one probabilistic choice. For a relation $\rightarrow \subseteq A \times \text{FDist}(A)$, we use

$$\text{NF}_{\rightarrow} = \{a \in A \mid \nexists \mu \in \text{FDist}(A). a \rightarrow \mu\}$$

for its normal forms. To follow the probability mass through several steps, it is lifted to a relation between multi-distributions. Normal forms are retained by a self-loop, so all lifted derivations can be viewed as infinite sequences. The following lifting is the standard multidistribution semantics of probabilistic rewriting [1, 3].

Definition 2.4.1: Lifting to multi-distributions

Let $\rightarrow \subseteq A \times \text{FDist}(A)$. Its *lifting* $\Rightarrow \subseteq \text{FDist}(A) \times \text{FDist}(A)$ is defined as follows. For

$$\mu = \{p_1 : a_1, \dots, p_k : a_k\},$$

choose, for every occurrence a_i , a multi-distribution ν_i such that

$$\nu_i = \{1 : a_i\} \quad \text{if } a_i \in \text{NF}_{\rightarrow}, \quad \text{or} \quad a_i \rightarrow \nu_i.$$

Then

$$\mu \Rightarrow \bigoplus_{i=1}^k p_i \cdot \nu_i.$$

Different legal steps may be chosen for different occurrences of the same term. Thus the lifting retains the nondeterminism of the underlying rewrite relation.

For a PTRS $\mathcal{P}$, the liftings of $\xrightarrow{\mathcal{P}}$, $\xrightarrow{\circ}_{\mathcal{P}}$, and $\xrightarrow{\iota}_{\mathcal{P}}$ are written $\xrightarrow{\mathcal{P}}$, $\xRightarrow{\circ}_{\mathcal{P}}$, and $\xRightarrow{\iota}_{\mathcal{P}}$, respectively.

In the running example, outermost rewriting gives the lifted derivation

$$\begin{aligned} \{1 : \text{first}(\text{produce})\} &\xrightarrow{\mathcal{P}_{\text{ex}}} \{\tfrac{1}{2} : \text{first}(\text{pair}(\text{a}, \text{produce})), \tfrac{1}{2} : \text{first}(\text{pair}(\text{b}, \text{produce}))\} \\ &\xrightarrow{\mathcal{P}_{\text{ex}}} \{\tfrac{1}{2} : \text{a}, \tfrac{1}{2} : \text{b}\}. \end{aligned}$$

Afterwards, both normal forms are retained. Under innermost rewriting, the second lifted step instead expands both nested occurrences of **produce**, and this process continues without producing normal-form probability mass.

Rewrite sequence trees provide an equivalent, and often more convenient, representation. Probabilistic alternatives are represented by branches, whereas the choice of a rule and a redex position is fixed locally at each internal node [3].

Definition 2.4.2: Rewrite sequence trees

Let $\rightarrow \subseteq A \times \text{FDist}(A)$. A *rewrite sequence tree* (RST) $\mathfrak{T}$ for $\rightarrow$ is a rooted, finitely branching tree whose nodes v are labelled by pairs $(p_v : a_v)$ with $0 < p_v \leq 1$. The root has probability 1. If a node v has children $w_1, \dots, w_k$, then

$$a_v \rightarrow \left\{ \frac{p_{w_1}}{p_v} : a_{w_1}, \dots, \frac{p_{w_k}}{p_v} : a_{w_k} \right\}.$$

Leaves are permitted to be reducible. The tree is *fully evaluated* if every leaf is labelled by a normal form. We write $N^{\mathfrak{T}}$ for the nodes and $\text{Leaf}^{\mathfrak{T}}$ for the leaves. The *convergence probability* of the tree is

$$|\mathfrak{T}| = \sum_{v \in \text{Leaf}^{\mathfrak{T}}} p_v.$$

In a partial RST, reducible leaves form an unfinished frontier, so their weight is included in $|\mathfrak{T}|$ even though they have not reached normal form. For a fully evaluated RST, all leaf weight is normal-form weight and $|\mathfrak{T}|$ directly represents the probability of termination.

An infinite branch contributes no leaf probability. Nevertheless, the total probability mass that never reaches a leaf can be zero even if infinite paths exist. This distinction is the reason why ordinary well-foundedness is too strong for probabilistic systems.

2.5 Probabilistic Termination

For a multi-distribution, the probability mass that has already reached a normal form is the following quantity.

Definition 2.5.1: Normal-form probability

Let $\rightarrow \subseteq A \times \text{FDist}(A)$ and $\mu \in \text{FDist}(A)$. Its *normal-form probability* is

$$|\mu|_{\rightarrow} = \sum_{\substack{(p:a) \in \mu \\ a \in \text{NF}_{\rightarrow}}} p.$$

Since the lifting retains normal forms, the sequence $|\mu_0|_{\rightarrow}, |\mu_1|_{\rightarrow}, \dots$ is non-decreasing along a lifted derivation and bounded by one. Its limit therefore exists.

Definition 2.5.2: Almost-sure termination [1]

A term $a \in A$ is *almost-surely terminating* with respect to $\rightarrow$ if every infinite lifted derivation

$$\mu_0 \Rightarrow \mu_1 \Rightarrow \mu_2 \Rightarrow \cdots, \quad \mu_0 = \{1 : a\},$$

satisfies

$$\lim_{n \rightarrow \infty} |\mu_n|_{\rightarrow} = 1.$$

The relation $\rightarrow$ is *almost-surely terminating*, written $\text{AST}(\rightarrow)$, if every $a \in A$ is almost-surely terminating.

Equivalently, $\text{AST}(\rightarrow)$ holds if every RST for $\rightarrow$ has convergence probability one; it suffices to consider fully evaluated RSTs.

This termwise formulation is equivalent to quantifying over infinite lifted derivations from arbitrary finite initial multi-distributions, since the finitely many weighted occurrences can be considered separately.

For a PTRS $\mathcal{P}$, *outermost AST* is $\text{AST}(\rightarrow_{\mathcal{P}})$, and *innermost AST* is $\text{AST}(\rightarrow_{\mathcal{P}}^{\downarrow})$. The quantification over every lifted derivation, or every RST, is essential. Probabilistic choices occur as branches inside a tree, while different rule or redex choices yield different trees. AST requires probability-one termination for all resolutions of this nondeterminism.

Example 2.5.3. The single-rule PTRS

$$\text{retry} \rightarrow \left\{ \frac{1}{2} : \text{done}, \frac{1}{2} : \text{retry} \right\}$$

has an infinite branch that chooses **retry** every time. After n steps, however, the probability mass still assigned to **retry** is 2^{-n} . Hence the normal-form probability tends to one, and the system is AST even though it is not terminating in the classical sense.

In Example 2.3.4, the term **first(produce)** is almost-surely terminating under outermost rewriting: all probability mass reaches **a** or **b** after two steps. Under innermost rewriting, its normal-form probability remains zero. This is the strategy gap that the transformation in Chapter 3 is designed to bridge.

Two stronger probabilistic termination notions are useful for distinguishing probability-one termination from finite expected runtime.

Definition 2.5.4: PAST and SAST [1, 3]

Let $\vec{\mu} = (\mu_n)_{n \in \mathbb{N}}$ be an infinite lifted derivation. Its *expected derivation length* is

$$\text{edl}(\vec{\mu}) = \sum_{n=0}^{\infty} (1 - |\mu_n|_{\rightarrow}).$$

The relation $\rightarrow$ is *positively almost-surely terminating*, written $\text{PAST}(\rightarrow)$, if $\text{edl}(\vec{\mu}) < \infty$ for every infinite lifted derivation with $\mu_0 = \{1 : a\}$ for some $a \in A$.

For $a \in A$, its *expected derivation height* is

$$\text{edh}_{\rightarrow}(a) = \sup \{ \text{edl}(\vec{\mu}) \mid \vec{\mu} = (\mu_n)_{n \in \mathbb{N}} \text{ is a lifted derivation and } \mu_0 = \{1 : a\} \}.$$

The relation is *strongly almost-surely terminating*, written $\text{SAST}(\rightarrow)$, if $\text{edh}_{\rightarrow}(a) < \infty$ for every $a \in A$.

The notions form the strict hierarchy

$$\text{SAST}(\rightarrow) \implies \text{PAST}(\rightarrow) \implies \text{AST}(\rightarrow),$$

in general [1, 3]. This thesis focuses on AST; PAST and SAST are included to make the chosen property precise and to prevent it from being confused with finite expected derivation length. PAST requires each nondeterministic derivation to have finite expected length; SAST additionally requires a uniform finite bound for all derivations from each fixed start term.

2.6 Transformation-Based Strategy Transfer

The transformation in this thesis is based on the deterministic construction of Thiemann [5]. Its extended signature contains fresh control symbols. Starting from a marked source term, the transformed system first moves a search marker towards a redex. Innermost rewriting permits the marker to descend only when the surrounding source term is not itself a redex; this enforces the outermost restriction. A transformed source rule then performs the actual rewrite step and changes the marker into a return marker. Finally, deterministic administrative rules move that marker back to the root before the next simulated step begins.

Theorem 2.6.1: Deterministic outermost-to-innermost transformation [5]

For every finite TRS $\mathcal{R}$ over a finite signature, the transformation constructs a finite TRS $\mathcal{R}'$ such that

$$\mathcal{R} \text{ is outermost terminating} \iff \mathcal{R}' \text{ is innermost terminating.}$$

Proof idea. — The forward simulation maps every outermost source step to a non-empty finite innermost derivation. For the converse, arbitrary infinite innermost derivations over the extended signature are analysed and a suitable derivation is projected back to an infinite outermost source derivation. The complete proof is given in [5].

An earlier transformation by Raffelsieper and Zantema instead reduces outermost ground termination to unrestricted termination. Its principal correctness result is one-way: termination of the transformed system implies outermost ground termination of the source. For finite quasi-left-linear TRSs, the required anti-matching information and the transformed system can be constructed finitely [4]. This result is useful as a second example of the simulation-based proof pattern, but it must not be confused with the outermost-to-innermost equivalence in Theorem 2.6.1.

These deterministic simulation results motivate the probabilistic construction, but they do not by themselves establish an AST result. The new proof must track probability mass through infinite lifted derivations or fully evaluated RSTs and must represent every nondeterministic outermost source choice. The concrete encoding, the transformed PTRS, and these correctness arguments are introduced in Chapter 3.

CHAPTER 3

[Title of Your Main Contribution]

This chapter contains the main theoretical (and possibly technical) contribution of the thesis. It should present a coherent body of new results, organized into sections.

For a theory-focused thesis: introduce new notions, state lemmas/theorems, provide proofs, and illustrate results with examples. For an implementation-focused thesis: additionally describe the algorithm, key design decisions, and data structures.

The opening paragraph of this chapter should cover the following points:

1. State what new objects, criteria, or techniques this chapter introduces.
2. Introduce a running example that will be used throughout to illustrate each new definition and result.
3. Outline the chapter structure by referencing the main sections.

3.1 [Central Definitions / The New Framework]

For self-introduced notions it is even more important to explain everything and used multiple examples to illustrate everything. Use one running example that can be used through the whole section and maybe some additional small examples to highlight special cases.

Definition 3.1.1: *[Name of Your First Definition]*

Example 3.1.2.

3.2 [Main Results]

Each result in this section should be presented as follows:

1. State and prove all auxiliary lemmas before the main theorem.
2. State the main theorem. Typical statements are:
 - **Soundness:** If the technique returns YES, then the property holds.
 - **Completeness:** If the property holds, the technique returns YES.
 - **Equivalence:** The new criterion holds if and only if an existing criterion holds.

3. Provide a complete proof, structured into numbered cases for long arguments. Reference definitions and earlier results by label.
4. Apply the theorem to the running example from Example 3.1.2, walking through every step explicitly.

Lemma 3.2.1: [*Name of Auxiliary Lemma*]

Proof.

□

Theorem 3.2.2: [*Name of Main Theorem, e.g., Soundness*]

Proof.

□

Example 3.2.3. Apply Theorem 3.2.2 to the running example from Example 3.1.2. Walk through every step of the argument explicitly.

CHAPTER 4

Evaluation

This chapter demonstrates that the approach from Chapter 3 works in practice.

A good evaluation chapter:

- Describes clearly what was implemented and where.
- Explains the experimental setup with enough detail to be reproducible.
- Presents results honestly, including cases where the technique does not help.
- Interprets the numbers and connects them back to the theory.

4.1 Implementation

The implementation section should cover:

1. The name of the tool extended and a brief description of its overall architecture (e.g., the pipeline from input parsing to proof output). and any external solvers used.
2. Which module(s) you added or changed and how they fit into the existing architecture.

4.2 Experimental Setup

The experimental setup should specify:

1. **Benchmark set.** Which set was used (e.g., TPDB, TermComp, SV-COMP, or a custom set)? How many examples total? Are there relevant subsets by category, strategy, or problem type?
2. **Configurations compared.** At minimum include the unmodified baseline and the tool with your extension. List any competing tools with a brief description.
3. **Hardware and time limits.** CPU model, amount of RAM, operating system, and per-example wall-clock timeout.

4.3 Results

The results section should:

1. Present results in a table (see Table 4.1).
2. Summarize the headline finding directly below the table, citing concrete numbers (additional YES answers, percentage improvement, overhead in seconds).
3. Point out interesting subsets or individual examples where the technique is especially effective and where the technique is not effective.
4. Note any surprising results to be discussed in Section 4.4.

Table 4.1 summarizes the results of our experiments.

Table 4.1: Experimental results on [*benchmark set name*]. *YES/NO* denotes the number of examples for which [*the property / its negation*] was proved. *MAYBE* means the tool returned no answer. *TO* means the timeout was exceeded. *Time* is the total wall-clock time in seconds.

Configuration	YES	NO	MAYBE	Avg. Time (s)
<i>Tool without extension</i>	–	–	–	–
<i>Tool with extension</i>	–	–	–	–

4.4 Discussion

The discussion should address the following questions:

1. Do the results match theoretical expectations from Chapter 3? If not, explain why.
2. On which types of examples does the technique help most? Identify structural properties of the benchmark examples that explain the gain.
3. Are there cases where the technique does not help or is slower? Explain the reason and relate it to the theoretical properties.
4. What are the main limitations and how could they be overcome? (This can bridge to the future work discussion in Chapter 5.)

Conclusion

The conclusion summarizes the thesis and discusses future work. Do not introduce any new material here. A reader who reads only the introduction and the conclusion should get a coherent picture of what was done and what remains open. It should follow the same structure as the introduction and the abstract, but with more emphasis on the bottom part of the hourglass.

The summary should address the following points:

1. Restate the problem studied and explain why it is relevant.
2. Describe the main contribution and the approach taken (reference Chapter 3).
3. Name the central theoretical result (reference Theorem 3.2.2) and state informally what it guarantees.
4. Summarize the experimental findings – number of additional examples solved, run-time overhead – if an evaluation was conducted (reference Chapter 4).

Each future work item should:

1. Identify a specific limitation of the current approach or a natural extension that was beyond the scope of this thesis.
2. Describe what would be needed to address it (new theoretical machinery, a different encoding, a richer formalism, etc.).
3. Explain what benefit this would bring (more examples solved, a stronger guarantee, applicability to a wider class of systems).

Bibliography

- [1] Martin Avanzini, Ugo Dal Lago, and Akihisa Yamada. On probabilistic term rewriting. *Science of Computer Programming*, 185:102338, 2020.
- [2] Franz Baader and Tobias Nipkow. *Term Rewriting and All That*. Cambridge University Press, Cambridge, UK, 1998.
- [3] Jan-Christoph Kassing and Jürgen Giesl. From innermost to full probabilistic term rewriting: Almost-sure termination, complexity, and modularity. *Logical Methods in Computer Science*, 21(4):28:1–28:79, 2025.
- [4] Matthias Raffelsieper and Hans Zantema. A transformational approach to prove outermost termination automatically. *Electronic Notes in Theoretical Computer Science*, 237:3–21, 2009. Proceedings of the 8th International Workshop on Reduction Strategies in Rewriting and Programming (WRS 2008).
- [5] René Thiemann. From outermost termination to innermost termination. In *SOFSEM 2009: Theory and Practice of Computer Science*, volume 5404 of *Lecture Notes in Computer Science*, pages 533–545, Berlin, Heidelberg, 2009. Springer.